

TRABAJO PRACTICO



“Filtrado Digital y FFT”

Ramiro Nieto Abascal

Nro. de legajo: 64.510

rnietoabascal@itba.edu.ar



Profesores a cargo: **Javier Petrucci**

jpetrucc@itba.edu.ar

Matias Bergerman

mbergerman@itba.edu.ar

Bruno Di Sanzo

bdisanzo@itba.edu.ar



Ayudantes: **Nicolás Agustín Beade**

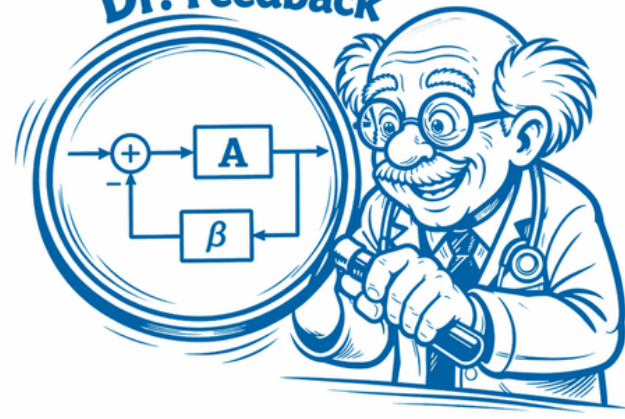
nbeade@itba.edu.ar



*Stability is not the absence of disturbance, but
the result of continuous correction*



Dr. Feedback



ÍNDICE

1. Introducción	3
1.1. Resumen	3
1.2. Presentación de los resultados	3
2. Ejercicio 1: Filtro anti-alias	4
2.1. Diseño de filtro	4
2.2. Implementación del sistema & verificación empírica del funcionamiento	6
2.3. Tiempo de procesamiento	8
3. Ejercicio 2: filtrado por convolución	9
3.1. Overlap and save	10
3.2. Implementación	12
4. Ejercicio 3: Analizador de THD	14
4.1. Diseño de bloques	14
4.1.1. Filtro comb	14
4.1.2. Pasa-bajos Recuperación de primeros 10 armónicos	15
4.1.3. Filtro pasabajos Recuperación de fundamental	17
4.1.4. Simulación	18
4.2. Análisis de implementación	20
4.2.1. Implementación en punto fijo	20
4.2.2. FFT como alternativa	21
4.3. Implementación en MCU	21

INTRODUCCIÓN

1.1 RESUMEN

El presente informe corresponde al Trabajo Práctico N°4: “Filtrado Digital y FFT”, de la materia Laboratorio de Electrónica III (25.24). A lo largo del documento se aborda el diseño de un filtro anti-alias para procesos de down-sampling, la optimización de la filtración mediante métodos de convolución segmentada (Overlap and Save) y, finalmente, el desarrollo de un analizador de Distorsión Armónica Total (THD) mediante estructuras de filtros tipo comb y filtros pasa-bajos para la recuperación de armónicos.

1.2 PRESENTACIÓN DE LOS RESULTADOS

En las siguientes secciones se presentan los resultados obtenidos durante el desarrollo de la práctica. En primera instancia, de corresponder, se hará mención del comportamiento esperado en función de las bases teóricas planteadas en clase. Luego, se hará una descripción de las mediciones realizadas y las conclusiones que se pueden extraer de estas. Finalmente, de corresponder, se detallarán propuestas de vías de investigación a futuro con tal de obtener un mejor entendimiento del contenido tratado.

EJERCICIO 1: FILTRO ANTI-ALIAS

Se desea realizar un down-sampling de $f_s = 10kHz$ a $f'_s = 2kHz$. La señal de interés está acotada en frecuencia hasta 970Hz; de este modo, con tal de evitar aliasing, se ha de diseñar un filtro pasa-bajos.

En vista al problema planteado, se solicita que la atenuación en 970Hz resulte de 6dB y una atenuación de al menos 50dB a partir de $f'_s/2 = 1000Hz$.

2.1 DISEÑO DE FILTRO

En función de los requerimientos previamente establecidos se diseña un filtro FIR mediante el método de la serie de Fourier. En este método resulta relevante la elección de la ventana a emplear en vista de optimizar el orden del filtro, donde varía, entre otras características, la atenuación del primer lóbulo lateral y la pendiente del lóbulo principal; afectando así el ancho de la banda de transición y la ganancia en la banda de atenuación. En vista a esta elección y en función de los requerimientos del filtro a implementar, se realizó una búsqueda entre las siguientes ventanas:

- Rectangular.
- Bartlett.
- Hann.
- Hamming.
- Blackman.

En función de este requerimiento, se implementó una búsqueda del número de taps necesario para cada una de ellas, con un máximo de 5000 (el proceso corresponde a buscar la función tal que la atenuación a partir de $f'_s/2 = 1000Hz$ sea de 50dB). Obteniendo así:

Tabla 2.1: Comparación de ventanas para el diseño del filtro FIR anti-alias

Ventana	Orden mínimo (taps)	Atenuación mínima [dB]	Cumple
Rectangular (boxcar)	–	–	No
Bartlett	–	–	No
Hann	761	50.10	Sí
Hamming	527	50.22	Sí
Blackman	741	50.21	Sí

donde además, se forzó en la búsqueda un número impar de taps tal que el filtro obtenido sea de fase lineal (por ende, group delay constante).

En función de la tabla 2.1 se decide emplear la ventana de Hamming en el diseño del filtro. Bajo esta configuración se obtiene la siguiente respuesta en frecuencia:

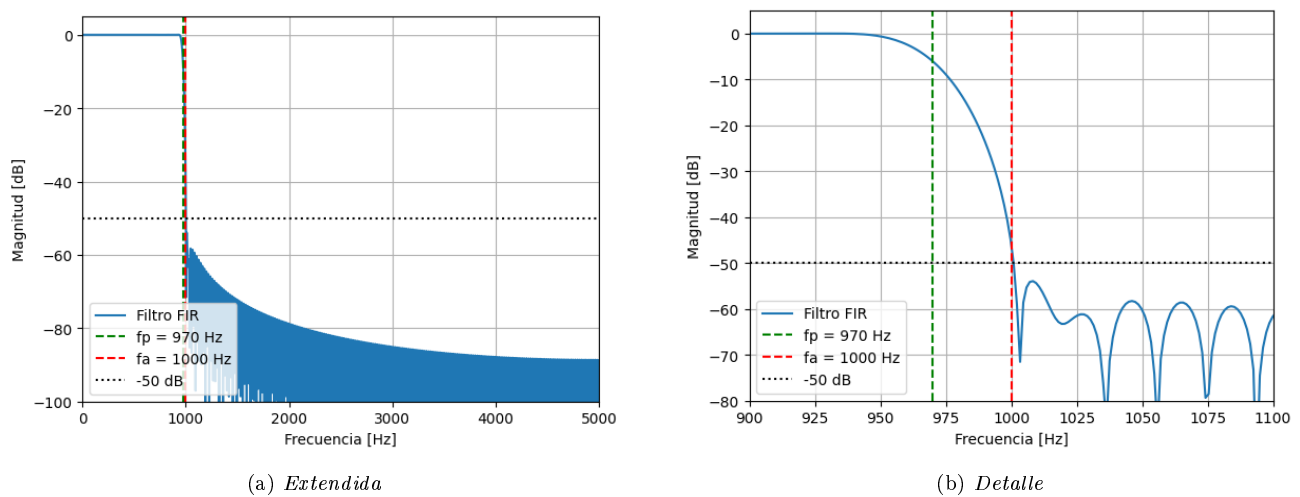


Figura 2.1: *Respuesta en frecuencia / Módulo*

Donde se verifica el comportamiento esperado (no se muestra un detalle en las inmediaciones de 970Hz dado que la forma en la que se implemento garantiza la atenuación de 6dB allí).

En tanto a otras características fundamentales del filtro se puede analizar la fase:

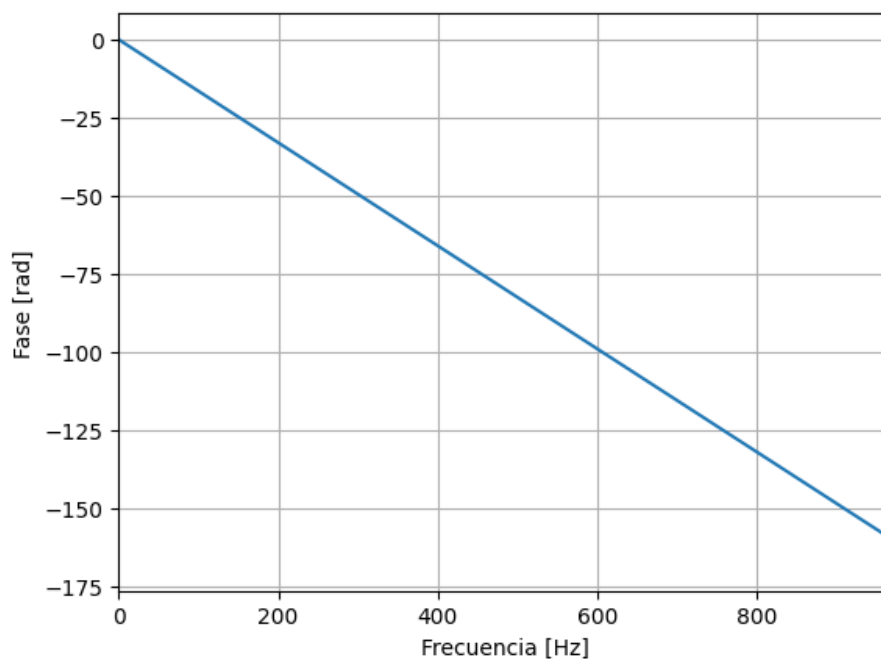


Figura 2.2: *Respuesta en frecuencia / Fase*

siendo relevante observar que efectivamente se dispone de fase lineal. Finalmente también resulta de interés observar la respuesta impulsiva del sistema:

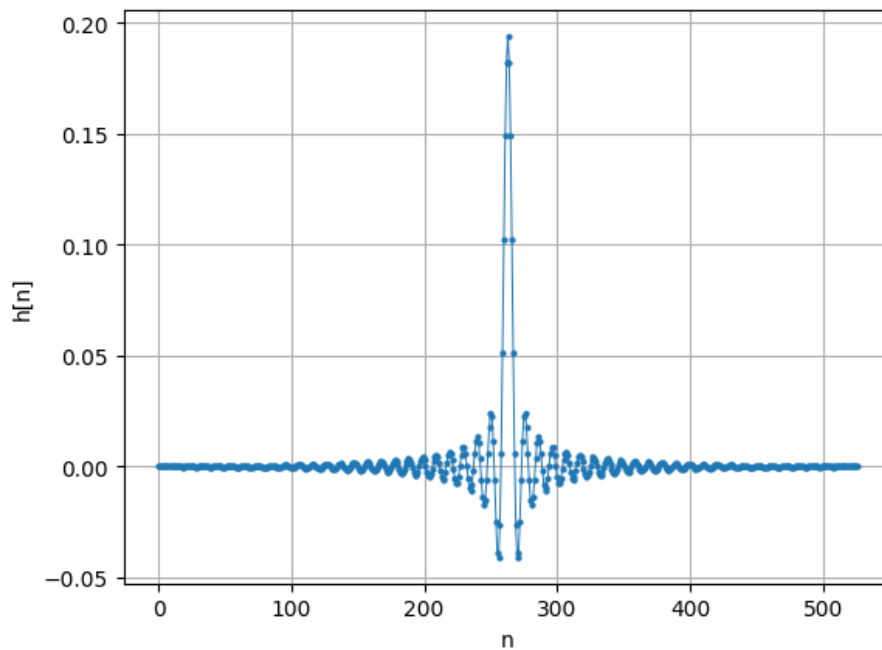


Figura 2.3: *Respuesta impulsiva.*

donde se observa la simetría para respecto a $N/2$.

2.2 IMPLEMENTACIÓN DEL SISTEMA & VERIFICACIÓN EMPÍRICA DEL FUNCIONAMIENTO

Se realizó la implementación del filtro:

```

1 def fir_filter(x, h):
2     x = np.asarray(x)
3     h = np.asarray(h)
4
5     N = len(h)
6     L = len(x)
7
8     y = np.zeros(L)
9
10    for n in range(L):
11        for k in range(N):
12            if n-k >= 0:
13                y[n] += h[k] * x[n-k]
14
15    return y

```

Listing 1: *Implementación del filtro.*

tras la cual puede realizarse el down-sampling deseado:

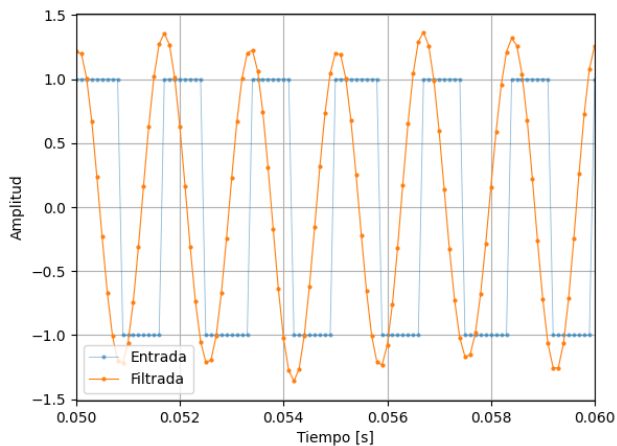
```

1 # --- filtrado ---
2 y = fir_filter(x, h)
3
4 #--- down-sampling ---
5 N_ds = 5
6
7 y_ds = y[::N_ds]
8 t_ds = t[::N_ds]

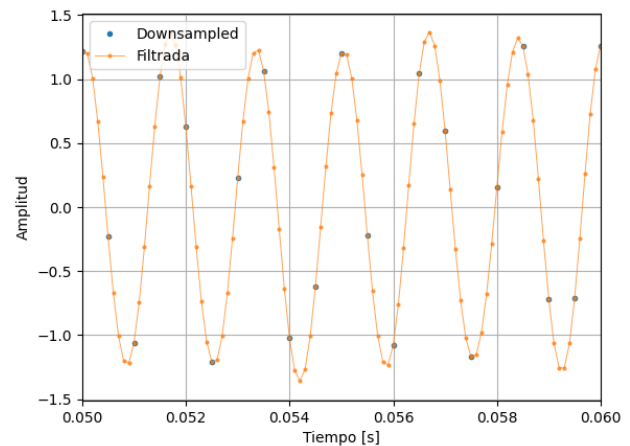
```

Listing 2: *Filtrado + down-sampling.*

En el marco temporal, estas operaciones corresponden con las siguientes señales:



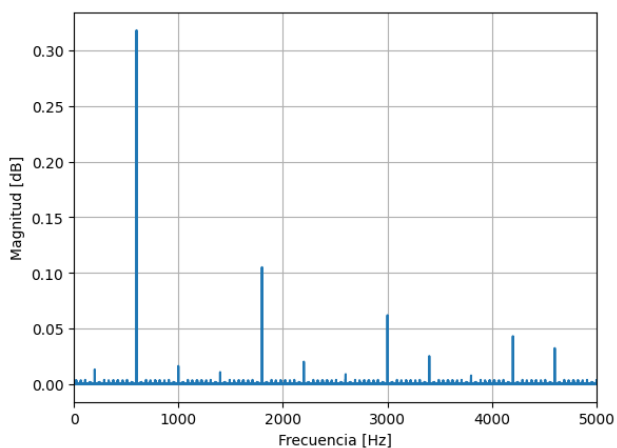
(a) *Entrada + señal filtrada.*



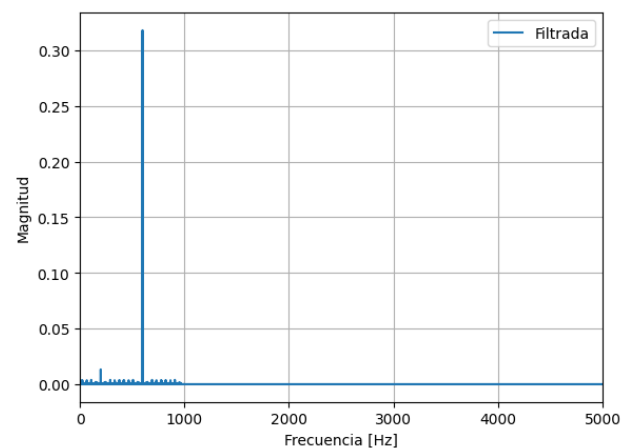
(b) *Señal filtrada + downsampling.*

Figura 2.4: Comparativa de señales en marco temporal

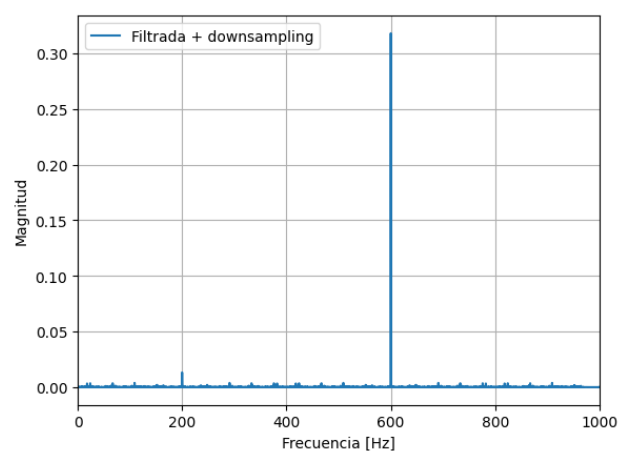
Esto se puede analizar desde el marco espectral realizando una FFT sobre las señales previamente indicadas:



(a) *Entrada*



(b) *Filtrada*



(c) *Filtrada + down-sampling*

Figura 2.5: Comparativa de señales en el dominio espectral

donde resulta relevante destacar que, por un lado, el cambio del rango en el eje corresponde a la modificación de la frecuencia de muestreo. Por otro lado, se observan armónicos que no eran esperados en función de la entrada indicada, estos se deben a que no se filtró la señal cuadrada a la hora de muestrearse, es decir, la entrada al sistema dispone de aliasing.

2.3 TIEMPO DE PROCESAMIENTO

Se solicita calcular el tiempo de procesamiento del sistema. Para ello se realizaron 5 ejecuciones del código previamente indicado obteniendo:

Tabla 2.2: *Tiempos de procesamiento del filtro FIR*

#(Ejecución)	Tiempo [s]	Tiempo por muestra [μ s]
1	19.135894	191.36
2	18.456400	184.56
3	18.152865	181.53
4	18.438790	184.39
5	18.219928	182.20
Promedio	18.480776	184.808

EJERCICIO 2: FILTRADO POR CONVOLUCIÓN

Se desea implementar el filtro desarrollado en la sección anterior mediante convolución. Este algoritmo tiene base en realizar una FFT de la entrada y respuesta impulsiva, para luego realizar el producto de las mismas en el marco espectral y anti-transformar (IFFT) el resultado, obteniendo así la salida del filtro:

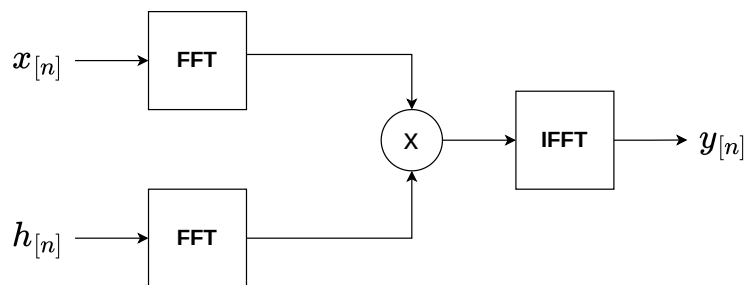


Figura 3.1: Esquema básico de filtrado por convolución

En función de que el algoritmo de FFT es $O(n \cdot \log(n))$ se busca optimizar el cálculo.

Si bien la operatoria parecería estar completamente definida bajo este esquema, si se requiere procesar la entrada en bloques se han de tener ciertas consideraciones. Para llegar a ellas debemos partir de la cuenta que se ha de realizar en el marco temporal. Si el filtro dispone de M coeficientes:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n - k]$$

lo cual corresponde con la operatoria habitual. Sin embargo, a la hora de emplear la DFT (o, en su defecto, la FFT) se realiza una extensión periódica de la señal, por lo que la operación desplazamiento no se condice con la expresión previamente indicada. A la operación que se debe realizar se la conoce como *convolución circular*, mientras que la que se realiza habitualmente lleva el nombre de *convolución lineal*. Esto parecería no tener relevancia para con la cuenta que se ha de efectuar en esta técnica, pues le compete al marco temporal y no al espectral (donde en principio tiene pie este proceso); sin embargo, esto es relevante para determinar la longitud de la FFT a realizar.

De querer filtrar una señal $x[n]$ de longitud L mediante el filtro previamente descrito ($h[n]$ de longitud M), la salida asociada tendría una longitud $L + M - 1$. De este modo, si se toma una cantidad de muestras N para realizar la FFT tal que sea menor que $L + M - 1$ el cálculo sería erróneo. Para visualizarlo tomemos un ejemplo sencillo:

$$x[n] = 1, 1$$

$$h[n] = 1, 1$$

aquí $L=M=2$. Si realizamos la convolución lineal y calculamos $y[0]$, observamos que:

$$y[0] = x[0] \cdot h[0] + x[1] \cdot h[-1] = 1 \cdot 1 + 1 \cdot 0 = 1$$

si ahora tomamos $N = 2$, la relación que se mantiene es la misma, pero la señal temporal asociada a la señal espectral obtenida con la DFT no tiene $h[-1] = 0$, sino que $h[1] = 1$. De este modo:

$$y[0] = x[0] \cdot h[0] + x[1] \cdot h[-1] = 1 \cdot 1 + 1 \cdot 1 = 2 \neq 1$$

de este modo, resulta evidente el problema que se estaba desarrollando. En vista a esto, la conclusión es que se ha de realizar un padding con ceros a la hora de calcular la FFT con tal de que la cuenta en el marco espectral se condiga con la cuenta deseada en el marco temporal. Consecuentemente se realiza una FFT de al menos $L + M - 1$ muestras. Siendo que la FFT es un algoritmo de cálculo de la DFT que resulta eficiente en con tamaños potencias de dos, es posible ajustar N tal que se den ambas condiciones simultáneamente. En este caso particular, siendo $L = 1024$ y $M = 527$, se puede tomar $N = 2048$.

Hasta aquí se concluyo la necesidad de añadir ceros adicionales para poder efectuar la cuenta correspondiente sin error. No obstante resta analizar dónde se han de colocar estos ceros, al igual que como se han de procesar los subsecuentes bloques. Existen dos técnicas que definen estos comportamientos:

- Overlap and save.
- Overlap and add.

En este informe se desarrollará el primero de ellos.

3.1 OVERLAP AND SAVE

Por un lado, la respuesta impulsiva a transformar dispone de $N-M$ ceros adicionales en su cola:

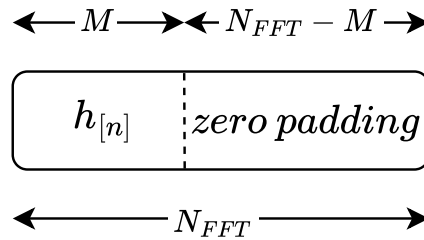


Figura 3.2: Respuesta impulsiva a transformar.

notar que la transformación de este bloque se efectúa una única vez (si se dispone de HW dedicado se puede almacenar únicamente esta transformada en lugar de los coeficientes de la respuesta impulsiva).

En tanto a la señal de entrada, se han de tomar también N muestras, siendo relevante estudiar cuales de ellas. En la primera iteración se colocan $M - 1$ ceros al comienzo del bloque, de lo contrario la cuenta efectiva con la FFT emplearía los elementos de la cola del bloque (recordar la extensión periódica que realiza esta operación). Luego de este padding se dispone de las primeras L muestras a procesar y se completa el bloque con ceros hasta alcanzar N muestras. En las siguientes iteraciones, al igual que cuando se realiza la convolución lineal, se han de tomar las $M-1$ muestras del bloque previo, junto con las siguientes L muestras a procesar + zero padding:

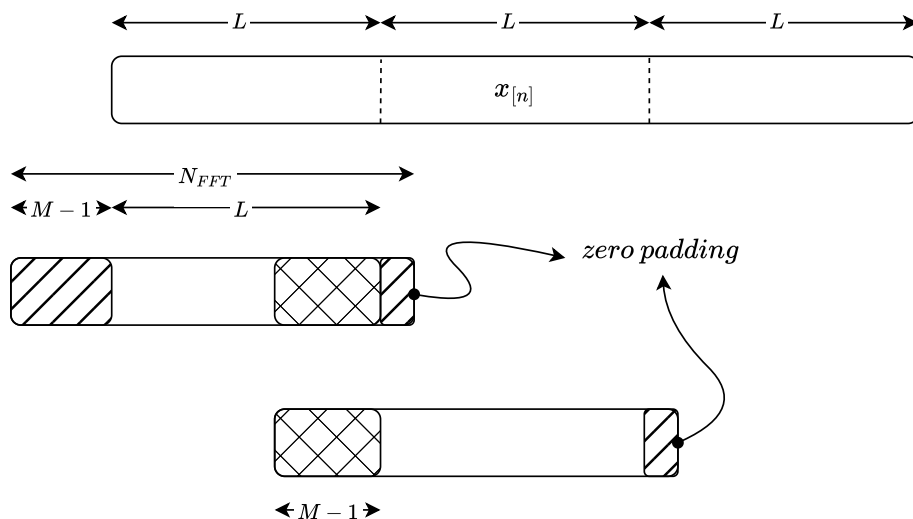


Figura 3.3: Armado de bloques de entrada.

Finalmente para con la salida, en cada iteración se obtiene N muestras a partir de la IFFT, de estas N muestras las primeras $M - 1$ no son válidas (por la misma razón que se comentó anteriormente), al igual que las últimas $N - (L - M - 1)$:

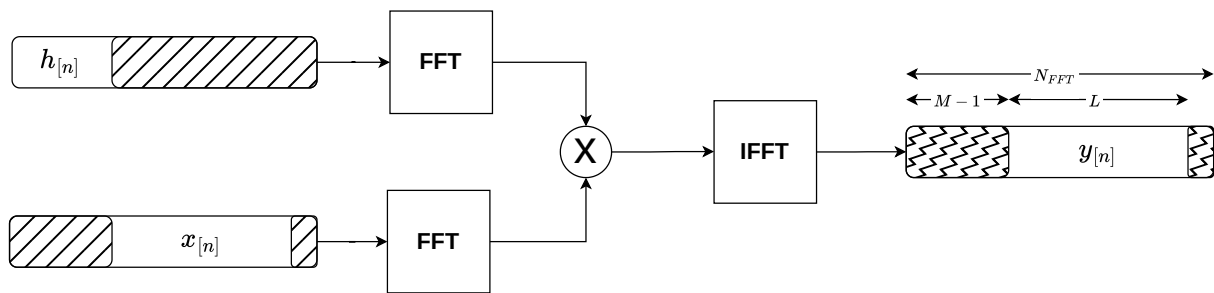


Figura 3.4: Diagrama de flujo.

3.2 IMPLEMENTACIÓN

Con tal de implementar el método desarrollado se hizo uso del siguiente código:

```

1 import numpy as np
2 import time
3
4 # --- Datos de entrada (asumiendo que ya existen) ---
5 # h: tus 527 coeficientes
6 # x: tu señal de entrada
7 # y_ref: el resultado de ejercicio 1 (convolucion lineal) para comparar
8
9 L = 1024
10 M = len(h)
11 N = 2048 # Potencia de 2 recomendada para h=527 y L=1024
12
13 # 1. Pre-cálculo: FFT del filtro (solo una vez)
14 H = np.fft.fft(h, n=N)
15
16 # Inicialización
17 salida_final = []
18 tiempos_bloque = []
19 pasado = np.zeros(M - 1) # M-1 ceros iniciales
20
21 # 2. Procesamiento por bloques
22 for i in range(0, len(x), L):
23     start_time = time.perf_counter()
24
25     # Tomamos el bloque actual de la señal
26     bloque_actual = x[i : i + L]
27
28     # Si el ultimo bloque es mas chico que L, lo rellenamos con ceros
29     if len(bloque_actual) < L:
30         bloque_actual = np.pad(bloque_actual, (0, L - len(bloque_actual)))
31
32     # Armado del bloque de entrada N=2048: [pasado(526) + actual(1024) +
33         # ceros(498)]
34     # Usamos np.concatenate o simplemente pasamos el tamaño a np.fft.fft
35     entrada_fft = np.concatenate([pasado, bloque_actual])
36
37     # Procesamiento espectral
38     X = np.fft.fft(entrada_fft, n=N)
39     Y = X * H
40     y_bloque = np.fft.ifft(Y).real # Nos quedamos con la parte real
41
42     # --- OVERLAP AND SAVE: Salvar desde el índice M-1 hasta (M-1 + L) ---
43     # Esto extrae exactamente las 1024 muestras limpias
44     bloque_limpio = y_bloque[M-1 : M-1+L]
45     salida_final.extend(bloque_limpio)
46
47     # Actualizamos el pasado para la siguiente iteración
48     pasado = bloque_actual[-(M-1):]
49
50     end_time = time.perf_counter()
51     tiempos_bloque.append(end_time - start_time)

```

```
51 # 3. Mediciones y Comparacion
52 salida_final = np.array(salida_final[:len(x)]) # Recortamos al largo original
53     si es necesario
54 tiempo_total = sum(tiempos_bloque)
55 tiempo_por_muestra = tiempo_total / len(x)
56
57 print(f"Tiempo total: {tiempo_total:.4f} s")
58 print(f"Tiempo por muestra: {tiempo_por_muestra:.8f} s")
59
60 # Comparacion con el ejercicio 1 (y_ref)
61 # Nota: y_ref debe ser recortado o comparado hasta len(x)
62 error = np.linalg.norm(salida_final - y_ref[:len(x)])
63 print(f"Error acumulado contra convolucion: {error}")
```

Listing 3: Código empleado en la implementación.

de este modo, se obtuvo un tiempo total de 38ms, equivalente a $0,38\mu s/muestra$ (con un error acumulado de 2×10^{-13} que se condice con un error computacional). Esta es una gran mejora con respecto a los tiempos obtenidos en el ejercicio 1 (18.48s de procesamiento netos + $184,8\mu s$). Esta gran diferencia se debe a, por un lado, la diferencia en la complejidad computacional ($O(n^2)$ vs $O(n \log(n))$), junto con un mejor empleo de los recursos (probablemente las funciones empleadas para el cálculo de la FFT e IFFT estén optimizadas, no así el código implementado en el ejercicio 1 con los dos for anidados).

EJERCICIO 3: ANALIZADOR DE THD

Se requiere diseñar un dispositivo de submetering que calcule en tiempo real el THD de la tensión de línea de una subestación eléctrica.

Para ello se ha de implementar el siguiente esquema:

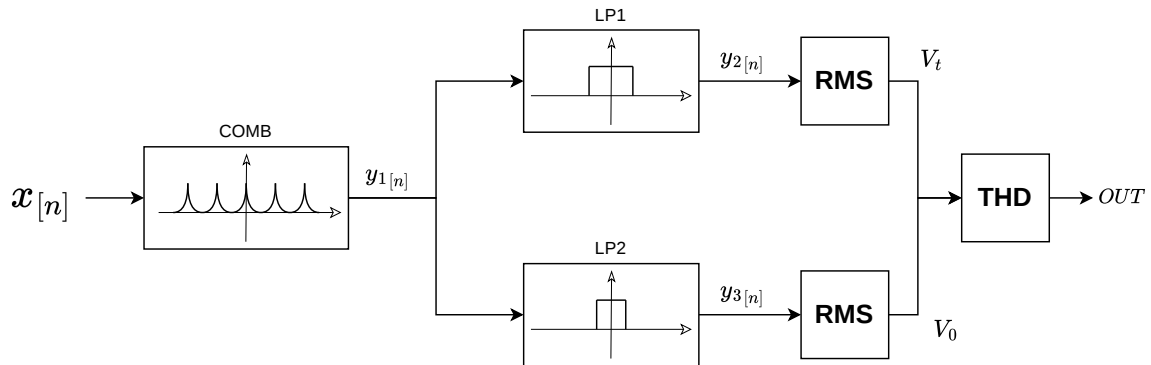


Figura 4.1: Esquema general a implementar.

4.1 DISEÑO DE BLOQUES

En este apartado se indicaran las características de cada uno de los bloques enseñados en la figura 4.1 y se realizara el diseño de los mismos.

4.1.1 FILTRO COMB

Dado que se desea analizar los armónicos de la red, debemos de diseñar un filtro que disponga de ganancia unitaria para estos y las componentes restantes. En función de ello se solicita la característica de atenuar al menos 10 veces las frecuencias múltiplo impar de 25Hz.

En vista de diseñar tal filtro, se dispone de la siguiente ecuación en diferencias:

$$y_1[n] = \alpha x[n] + \beta y_1[n - k] \quad (1)$$

donde hemos de determinar α , β y k . Para ello podemos realizar un análisis desde la función transferencia del sistema:

$$H(z) = \frac{Y_1(z)}{X(z)} = \frac{\alpha}{1 - \beta z^{-k}} = \frac{\alpha \cdot z^k}{z^k - \beta} \quad (2)$$

donde resulta evidente que se dispone de k ceros en el origen y k polos repartidos uniformemente en una circunferencia de radio $\beta^{1/k}$. De este modo, k ha de corresponder con la cantidad de armónicos de 50Hz que entran en $f_s = 10kHz$; de este modo, $k=200$. Luego debemos de ajustar

α y β tal de obtener las ganancias deseadas. En función de ello se puede analizar la respuesta en frecuencia del sistema:

$$|H(\omega)| = \frac{\alpha}{\sqrt{(\cos(\omega k) - \beta)^2 + \sin^2(\omega k)}} = \frac{\alpha}{\sqrt{1 + \beta^2 - 2 \cdot \beta \cdot \cos(\omega k)}} =$$

de este modo, la ganancia en los picos esta dada cuando $\omega = 2\pi/k$, tal que:

$$\frac{\alpha}{1 - \beta} = 1 \implies \alpha = 1 - \beta \quad (3)$$

mientras que los múltiplos impares de 25Hz corresponden a $\omega = \frac{2\pi \cdot m \cdot 25Hz}{2\pi \cdot f_s} = m \cdot \frac{\pi}{k}$ con m impar. Por ende:

$$\frac{\alpha}{1 + \beta} = 0,1 \implies \alpha = 0,1 + 0,1\beta \quad (4)$$

resolviendo el sistema de ecuaciones se obtiene $\alpha = 2/11$ y $\beta = 9/11$.

En función de los parámetros obtenidos puede graficarse la respuesta en frecuencia asociada:

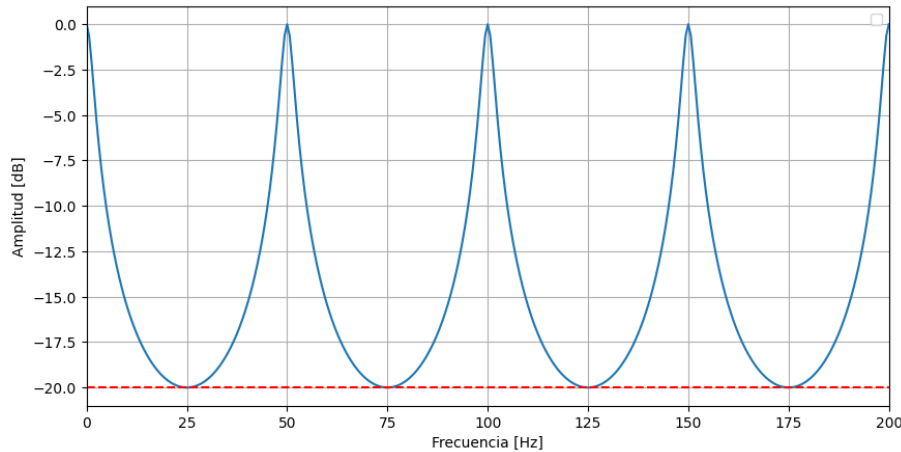


Figura 4.2: Módulo de respuesta en frecuencia de filtro comb.

4.1.2 PASA-BAJOS | RECUPERACIÓN DE PRIMEROS 10 ARMÓNICOS

En segundo lugar se debe diseñar un filtro pasa-bajos tal que se recuperen los primeros 10 armónicos, donde la atenuación subsecuente ha de ser de al menos 20dB. Para lograr este acometido se ha de diseñar un filtro FIR mediante el método de la serie de Fourier (análogo al ejercicio 1). En función de ello se ha de decidir la ventana óptima para la tarea.

En este caso particular la decisión es más compleja que las circunstancias planteadas para el ejercicio 1. Por un lado se ha de considerar que la ganancia en la banda de atenuación no es tan restrictiva como lo fue en el primer caso. Una atenuación de 20dB es lo suficientemente pequeña tal que se pueda emplear una ventana rectangular, por lo que potencialmente se puede aprovechar

la gran caída de la misma para optimizar el número de taps que ha de disponer el filtro. Sin embargo, esta ventana dispone de un problema fundamental: el fenómeno de Gibbs. La respuesta en frecuencia presenta oscilaciones en la banda pasante. Estas oscilaciones potencialmente pueden llegar a ser críticas para el cálculo del THD, dado que en el esquema de la Figura 4.1 no se dispone de un método para discernir de los armónicos y compensar esta atenuación. En función de ello se puede decidir optar por otra ventana que no disponga de esta problemática, como puede ser una ventana de Hamming. Esta ventana no dispone de estas oscilaciones pronunciadas, sin embargo el número de taps a emplear potencialmente crece drásticamente respecto de la primera opción. En función de ello se sugiere el uso de una tercera opción que no fue analizada en el primer ejercicio: la ventana de Kaiser. Esta ventana dispone de un parámetro adicional (β), que le confiere un grado de libertad que permite ajustar la caída y atenuación del lóbulo lateral. De este modo podemos intentar encontrar un punto intermedio entre los beneficios que confieren las ventanas previamente mencionadas, optimizando para nuestro beneficio.

En función de ello se realizó un código en Python tal que se trimmeo el número de taps y β (en el caso de la ventana de Kaiser), con tal de cumplir las especificaciones requeridas. Para la frecuencia de cruce que requiere como parámetro de entrada la función `signal.firwin()` se empleó $f_c=525\text{Hz}$ (notar que si se usa 500Hz potencialmente se estaría incurriendo en un error al atenuar un armónico relevante para el cálculo del THD). De este modo se obtuvieron:

Tabla 4.1: *Parámetros de diseño finales para el filtro pasabajos $y_2[n]$ ($f_c = 530\text{Hz}$)*

Ventana	Orden (N)	Parámetro Extra	Retardo de Grupo ($N/2$)
Rectangular	221	-	110.5 muestras
Hamming	451	-	225.5 muestras
Kaiser	281	$\beta = 1,8$	140.5 muestras

que se condice con las siguientes respuestas en frecuencia:

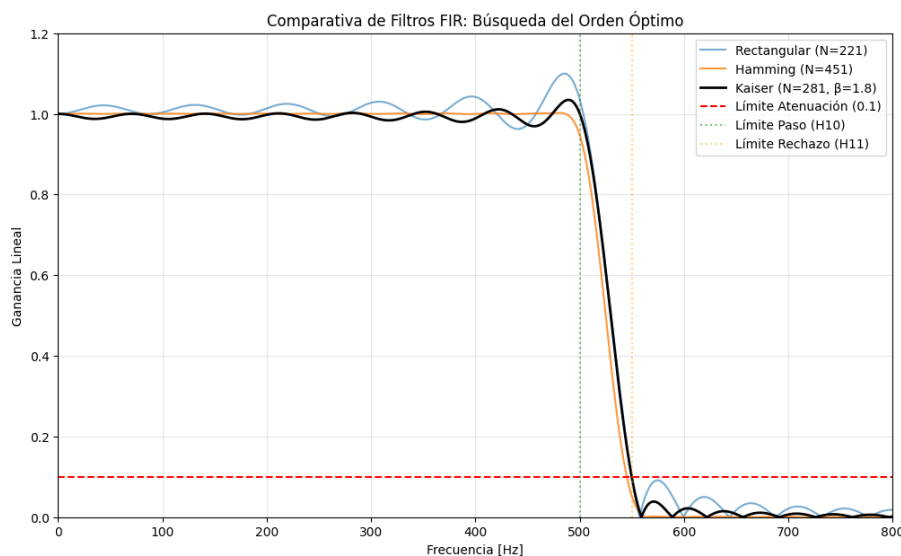


Figura 4.3: *Módulo de respuesta en frecuencia de filtro pasa-bajos.*

En este contexto es relevante las ganancias en las frecuencias de interés:

Tabla 4.2: Comparativa de ganancia lineal en los armónicos según el tipo de ventana ($f_s = 10\text{kHz}$)

Armónico	Frecuencia [Hz]	Rectangular	Hamming	Kaiser
H1	50	1.0201	1.0001	0.9921
H2	100	1.0036	1.0005	0.9880
H3	150	1.0131	1.0008	0.9985
H4	200	1.0136	1.0007	0.9976
H5	250	1.0008	1.0004	0.9860
H6	300	1.0274	0.9998	0.9931
H7	350	0.9862	0.9995	1.0049
H8	400	1.0420	0.9998	0.9885
H9	450	0.9733	1.0004	0.9735
H10	500	1.0366	0.9478	1.0013
H11	550	0.0998	0.0524	0.0986
H12	600	0.0020	0.0002	0.0205

de donde es posible extraer el error que se está cometiendo debido a las oscilaciones y/o la ganancia en el último armónico:

Tabla 4.3: Análisis de error porcentual en la banda de paso por efecto de rizado

Armónico	Err. Rect [%]	Err. Hamm [%]	Err. Kais [%]
H1	2.01	0.01	0.79
H2	0.36	0.05	1.20
H3	1.31	0.08	0.15
H4	1.36	0.07	0.24
H5	0.08	0.04	1.40
H6	2.74	0.02	0.69
H7	1.38	0.05	0.49
H8	4.20	0.02	1.15
H9	2.67	0.04	2.65
H10	3.66	5.22	0.13
Error Máximo	4.20	5.22	2.65
Error Medio	1.98	0.56	0.89

de este modo se optó por emplear la ventana de Kaiser que muestra un comportamiento más estable en ambas características que las otras dos opciones.

4.1.3 FILTRO PASABAJOS | RECUPERACIÓN DE FUNDAMENTAL

En última instancia se debe de diseñar un filtro pasa-bajos con tal de recuperar el armónico fundamental. Aquí se dispone de la misma problemática que en el caso anterior; sin embargo se dispone de la ventaja de que idóneamente se dispondrá de un único armónico, por lo que es posible compensar la ganancia del fenómeno de Gibbs que provoca la ventana rectangular. No obstante, se ha de evitar tener cambios muy abruptos ante posibles variaciones en la frecuencia de la red. De este modo se optó por emplear la misma ventana de Kaiser, ajustando el número de taps a 281, y un β de 2. Obteniendo así la siguiente respuesta en frecuencia:

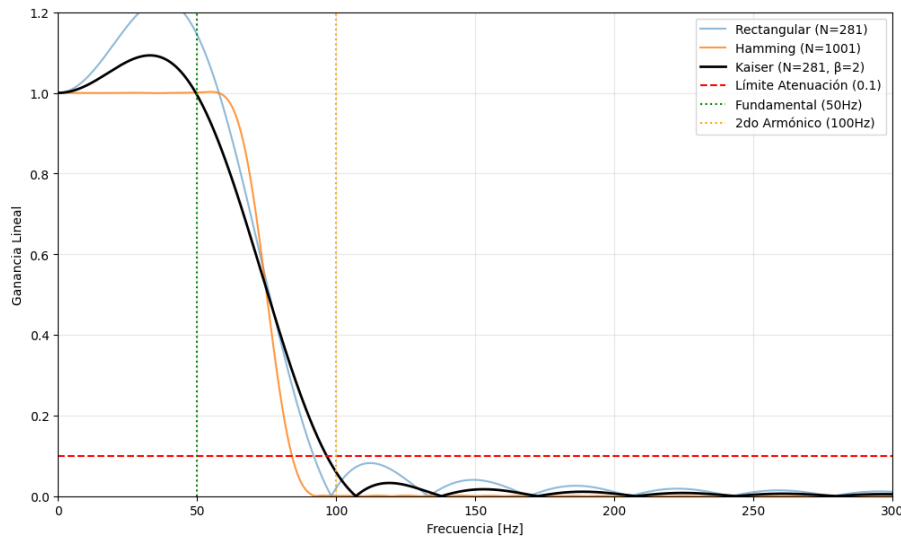


Figura 4.4: Módulo de respuesta en frecuencia de filtro pasa-bajos / Extracción de fundamental.

4.1.4 SIMULACIÓN

En última instancia se pide implementar el esquema completo y simularlo con la siguiente señal:

$$x[n] = A_1 \cdot \sin(2\pi \cdot n \cdot f_0 / f_s) + \sum_{k=2}^K A_k \cdot \sin(2\pi k f_0 \cdot n f_s) \quad (5)$$

donde $f_0 = 50Hz$, $f_s = 10kHz$ y A_k es el k-esimo número de la sucesión de Fibonacci.

El THD de esta señal se puede calcular como:

$$THD = \frac{\sum_{k=2}^K 1/2 \cdot A_k^2}{1/2 \cdot A_1^2} \quad (6)$$

por lo que podemos emplear esta misma para validar el sistema diseñado hasta aquí. Para ello se realizó una simulación, donde se generó un segundo de esta señal y se la inyectó al sistema. Una vez que se acabó el transitorio se midieron los valores RMS de la señal de salida y se calculó el THD como:

$$THD_{medido} = \frac{\sqrt{V_t^2 - V_0^2}}{V_0} \quad (7)$$

Tabla 4.4: Resultados finales de la simulación del analizador de THD

Parámetro	Valor obtenido
Tensión RMS total (V_t - 10 armónicos)	0.8366
Tensión RMS fundamental (V_0)	0.7026
THD calculado por el algoritmo	64.64 %
THD teórico (Armónicos H_2 a H_{10})	65.29 %
Error relativo porcentual	0.99 %

donde el error observado esta asociado a las oscilaciones en la banda pasante de los filtros diseñados.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 # --- 1. Parametros Globales ---
6 fs = 10000
7 duration = 1.0 # 1 segundo
8 t = np.arange(0, duration, 1/fs)
9 f0 = 50
10
11 # --- 2. Generacion de Senal Sintetica (Fibonacci) ---
12 def fibonacci_amplitudes(n_harmonics):
13     fib = [1, 1]
14     for i in range(2, n_harmonics + 2):
15         fib.append(fib[-1] + fib[-2])
16     # Ak = 1 / f_{k+1}
17     return [1/fib[k] for k in range(1, n_harmonics + 1)]
18
19 # Generamos armonicos hasta Nyquist (fs/2 = 5000Hz -> 100 armonicos)
20 K = 100
21 A = fibonacci_amplitudes(K)
22 x = np.zeros_like(t)
23
24 for k in range(1, K + 1):
25     x += A[k-1] * np.sin(2 * np.pi * k * f0 * t)
26
27 # --- 3. Filtro Comb (y1) ---
28 # y1[n] = alpha*x[n] + beta*y1[n-k]
29 alpha = 2/11
30 beta = 9/11
31 k_delay = 200
32
33 # Implementacion recursiva del Comb
34 # b = [alpha], a = [1, 0, 0, ..., -beta]
35 b_comb = [alpha]
36 a_comb = np.zeros(k_delay + 1)
37 a_comb[0] = 1
38 a_comb[k_delay] = -beta
39 y1 = signal.lfilter(b_comb, a_comb, x)
40
41 # --- 4. Filtros FIR Kaiser (y2 e y3) ---
42 # y2: Recupera 10 armonicos
43 n2 = 281
44 beta2 = 1.8
45 fc2 = 530
46 taps_y2 = signal.firwin(n2, fc2, fs=fs, window=('kaiser', beta2))
47 y2 = signal.lfilter(taps_y2, [1.0], y1)
48
49 # y3: Recupera fundamental
50 n3 = 281
51 beta3 = 2.0
52 fc3 = 75
53 taps_y3 = signal.firwin(n3, fc3, fs=fs, window=('kaiser', beta3))
54 y3 = signal.lfilter(taps_y3, [1.0], y1)
55
56 # --- 5. Calculo de THD ---

```

```

57 # Para evitar el transitorio inicial de los filtros, descartamos el primer
    tramo
58 offset = 5000
59 y2_steady = y2[offset:]
60 y3_steady = y3[offset:]
61
62 # Valores RMS
63 Vt = np.sqrt(np.mean(y2_steady**2))
64 V0 = np.sqrt(np.mean(y3_steady**2))
65
66 # THD = sqrt(Vt^2 - V0^2) / V0
67 thd_calculado = (np.sqrt(Vt**2 - V0**2) / V0) * 100
68
69 # --- 6. Validacion Teorica ---
70 # THD_teorico usando los primeros 10 armonicos (A1 a A10)
71 sum_sq_harmonics = np.sum(np.array(A[1:10])**2)
72 thd_teorico = (np.sqrt(sum_sq_harmonics) / A[0]) * 100
73
74 print(A)
75 print(f"--- Resultados del Analizador ---")
76 print(f"V_total (10 armonicos): {Vt:.4f}")
77 print(f"V_fundamental (V0): {V0:.4f}")
78 print(f"THD calculado: {thd_calculado:.2f}%")
79 print(f"THD teorico (H2-H10): {thd_teorico:.2f}%")
80
81 # --- 7. Visualizacion ---
82 plt.figure(figsize=(12, 5))
83 plt.plot(t[offset:offset+1000], y2[offset:offset+1000], label='y2 (10
    Armonicos)')
84 plt.plot(t[offset:offset+1000], y3[offset:offset+1000], label='y3 (
    Fundamental)', linewidth=2)
85 plt.legend()
86 plt.grid(True)
87 plt.show()

```

Listing 4: Código empleado en la implementación.

4.2 ANÁLISIS DE IMPLEMENTACIÓN

Si bien el sistema diseñado cumple con las especificaciones requeridas, existen ciertos riesgos a la hora de realizar la implementación.

4.2.1 IMPLEMENTACIÓN EN PUNTO FIJO

El cálculo del valor RMS de una señal involucra la operación:

$$V_{RMS} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} y[n]^2} \quad (8)$$

La presencia de la raíz cuadrada y la división por N provocan que inherentemente la sumatoria de los cuadrados se desvíe ampliamente del rango de la señal original y el valor RMS a calcular. Por ende, este da origen a un riesgo de overflow al implementarlo en punto fijo si no se toman las precauciones necesarias.

Una forma de mitigar este riesgo es realizar un cálculo del promedio móvil de los cuadrados, es decir, en lugar de calcular la sumatoria de los cuadrados de toda la señal, se puede calcular la sumatoria de los cuadrados de un bloque de muestras y luego dividir por el número de muestras

en ese bloque. De este modo, se mantiene el valor dentro de un rango manejable y se reduce el riesgo de overflow:

$$MS_n = MS_{n-1} + \frac{y[n]^2 - MS_{n-1}}{n} \quad (9)$$

4.2.2 FFT COMO ALTERNATIVA

Se sugiere el uso de la FFT como alternativa para el cálculo del THD, empleando la amplitud de los armónicos asociados para determinar los valores RMS.

Este método dispone de una ventaja de cálculo con respecto a lo observado en la implementación propuesta (notar el orden de los filtros diseñados); donde inclusive se podría emplear el algoritmo de Goertzel (algoritmo óptimo para el cálculo de bins individuales de la DFT) con tal de optimizar aun más el cálculo de los armónicos de interés. No obstante, esta alternativa dispone de un riesgo asociado: ¿Qué ocurre si los armónicos no coinciden exactamente con los bins de la FFT?

En un principio, dado que se conoce de antemano el contenido espectral que se desea medir, es posible tomar una ventana adecuada para minimizar el leakage y así obtener una medición precisa de los armónicos. Sin embargo, a diferencia del filtro comb implementado, esta alternativa no evita que el contenido espectral de la señal de entrada que esta en las inmediaciones de los armónicos de interés se asocie al bin calculado, por lo que este leakage es potencialmente riesgoso (aunque si la señal sensada es lo suficientemente limpia no debería de ser crítica esta cuestión).

Finalmente, se ha de tener en cuenta que ocurriría si la frecuencia fundamental no corresponde exactamente con 50Hz (dado que el diseño del procedimiento previamente descrito involucra esta cuestión). De ser así los armónicos a estudiar no coincidirán con los bins de la FFT, por lo que la amplitud de los mismos variaría en correspondencia con la diferencia de frecuencia y la ventana empleada. Sin embargo, notar que en el método implementado no estamos exceptuados de esta problemática, en tanto y en cuanto el filtro comb tiene un ancho de banda acotado, por lo que fluctuaciones de la fundamental alteran también la amplitud de la señal obtenida, aunque en principio es menos crítico.

4.3 IMPLEMENTACIÓN EN MCU

Se dispusieron de algunos inconvenientes a la hora de realizar la implementación de este sistema en una MCU. El código empelado corresponde a:

```

1 /*
2  * main.c
3  *
4  *   Created on: Jun 23, 2025
5  *   Author: mpaaurabe
6  */
7
```

```

8  #include "fsl_device_registers.h"
9  #include "hardware.h"
10 #include "PingPongBuffer.h"
11 #include <ports.h>
12 #include "config.h"
13 #include <math.h>
14
15 #define BUFFER_SIZE 1000
16 #define SAMPLE_RATE_HZ 10000U
17
18
19 volatile bool nextsample;
20
21 volatile uint16_t *adc_in;
22 volatile uint16_t *dac_out;
23 static int alternate = -1;
24
25 void my_app(void);
26
27 int main(void)
28 {
29     hw_Init();
30     Init_Ports();
31     PingPongBuffer_Init(SAMPLE_RATE_HZ, BUFFER_SIZE);
32
33     for(;;)
34     {
35         if(SW2_read())
36             alternate = -1;
37
38         if(SW3_read())
39             alternate = 1;
40
41
42         if(nextsample == 1)
43         {
44             my_app();
45             nextsample = 0;
46         }
47     }
48 }
49
50
51
52 #define FIR_TAP1      281
53 #define FIR_TAP2      281
54 #define COMB_DELAY    200
55
56 const float taps_FIR1[281] = {...};
57 const float taps_FIR2[281] = {...};
58
59 void FIR_Process_Buffer(float *in, float *out, int numElem, float *state, int
    *pIdx, const float *taps, int numTaps);
60 float RMS_calculator(float *s, int numElem);
61
62 // Estados para los filtros
63 static float state_comb[COMB_DELAY] = {0};
64 static int idx_comb = 0;
65
66 static float state_y2[FIR_TAP1] = {0};
67 static int idx_y2 = 0;
68
69 static float state_y3[FIR_TAP2] = {0};
70 static int idx_y3 = 0;
71
72 // Parametros del Comb
73 const float alpha = 0.18181f; // 2/11
74 const float beta  = 0.81818f; // 9/11
75
76 void my_app(void)
77 {
78     static float modul = 0.5;
79     volatile uint16_t *adc_in = GetBufferIn();
80     volatile uint16_t *dac_out = SetBufferOut();
81

```

```

82     TurnON(TP0);
83
84     // Senales a implementar
85     static float y1[BUFFER_SIZE];
86     static float y2[BUFFER_SIZE];
87     static float y3[BUFFER_SIZE];
88
89     int i,j;
90
91     for (int i = 0; i < BUFFER_SIZE; i++)
92     {
93         float x_n = (float)adc_in[i] - 2048.0f;
94
95         // Leemos el valor que tiene el delay (la salida se hace D muestras)
96         volatile float delayed_y = state_comb[idx_comb];
97
98         // Calculamos la salida actual siguiendo  $y[n] = a \cdot x[n] + b \cdot y[n-D]$ 
99         y1[i] = (alpha * x_n) + (beta * delayed_y);
100
101         // Guardamos la salida actual en el buffer de estado para que sea
102         // el "delayed_y" dentro de D muestras
103         state_comb[idx_comb] = y1[i];
104
105         // Avanzamos el puntero circular
106         idx_comb++;
107         if (idx_comb >= COMB_DELAY) idx_comb = 0;
108     }
109
110     FIR_Process_Buffer(y1, y2, BUFFER_SIZE, state_y2, &idx_y2, taps_FIR1,
111                       FIR_TAP1);
112     FIR_Process_Buffer(y1, y3, BUFFER_SIZE, state_y3, &idx_y3, taps_FIR2,
113                       FIR_TAP2);
114
115     float V0 = RMS_calculator(y3, BUFFER_SIZE);
116     float Vt = RMS_calculator(y2, BUFFER_SIZE);
117
118     float THD = sqrtf(Vt * Vt - V0 * V0)/V0;
119
120     for (int i = 0; i < BUFFER_SIZE; i++)
121         dac_out[i] = (uint16_t)(THD * 4095.0f);
122
123     modul *= alternate;
124     TurnOFF(TP0);
125 }
126
127 /**
128  * @brief Procesa un buffer completo de entrada y genera uno de salida.
129  * @param in Puntero al buffer de entrada (ej: y1_buffer)
130  * @param out Puntero al buffer donde se guardara la salida (ej:
131  *           y2_buffer)
132  * @param numElem Tamano del buffer (BUFFER_SIZE, ej: 1024)
133  * @param state Array de historial (debe ser estatico y de tamano numTaps)
134  * @param pIdx Puntero al indice actual del buffer circular (mantiene el
135  *            estado entre bloques)
136  * @param taps Array de coeficientes
137  * @param numTaps Cantidad de coeficientes (281)
138  */
139 void FIR_Process_Buffer(float *in, float *out, int numElem, float *state, int
140 *pIdx, const float *taps, int numTaps)
141 {
142     // Bucle que recorre cada muestra del buffer de entrada
143     for (int i = 0; i < numElem; i++)
144     {
145         // Guardar la muestra actual en el buffer circular de estado
146         state[*pIdx] = in[i];
147
148         // Convolucion: Multiplicar y Acumular (MAC)
149         float acc = 0.0f;
150         int currIdx = *pIdx;
151
152         for (int j = 0; j < numTaps; j++)
153         {

```

```

150         acc += taps[j] * state[currIdx];
151
152         // Retroceder en el buffer circular
153         currIdx--;
154         if (currIdx < 0) {
155             currIdx = numTaps - 1;
156         }
157     }
158
159     // Escribir el resultado en el buffer de salida
160     out[i] = acc;
161
162     // Avanzar el puntero del buffer circular para la PROXIMA muestra
163     *pIdx = (*pIdx + 1) % numTaps;
164 }
165 }
166
167
168 float RMS_calculator(float *s, int numElem)
169 {
170     float acc = 0.0f;
171     for (int i = 0; i < numElem; i++)
172     {
173         float val = s[i];
174         acc += val * val;
175     }
176
177     return sqrtf(acc / (float)numElem);
178 }

```

Listing 5: Implementación en MCU

En vista a debuggear este código se verificó que el tiempo de cálculo de los filtros no supere el tiempo de adquisición de cada bloque. Este ensayo se realizó mostrando a la salida la entrada, donde se pudo observar la entrada sin distorsión alguna, por lo que se concluye que el tiempo de cálculo es menor al tiempo de adquisición.

Con tal de verificar que la secuencia empleada no sea errónea, se procedió a generar mediante python una señal de prueba, junto con el set de *golden values* asociados:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 # --- Configuración idéntica al código C ---
6 BUFFER_SIZE = 1000
7 SAMPLE_RATE = 10000 # 10 kHz
8 COMB_DELAY = 200
9 ALPHA = 0.18181
10 BETA = 0.81818
11
12 def generate_inputs(filename, duration_sec=1.0):
13     """Genera una señal de prueba: Seno de 50Hz + 3er armónico + Ruido"""
14     t = np.arange(0, duration_sec, 1/SAMPLE_RATE)
15
16     # Amplitudes para el cálculo del THD esperado
17     # Fundamental (50Hz) y 3er Armónico (150Hz)
18     amp_fund = 1000.0
19     amp_harm = 300.0
20
21     fundamental = amp_fund * np.sin(2 * np.pi * 50 * t)
22     harmonico = amp_harm * np.sin(2 * np.pi * 150 * t)
23     ruido = 50 * np.random.normal(size=len(t))
24
25     signal_float = 2048 + fundamental + harmonico + ruido
26     signal_uint16 = np.clip(signal_float, 0, 4095).astype(np.uint16)
27
28     np.savetxt(filename, signal_uint16, fmt='%d')

```



```

29     print(f"Archivo de entrada '{filename}' generado.")
30
31     # Retornamos la senal y el THD teorico (armonico / fundamental)
32     thd_teorico = amp_harm / amp_fund
33     return signal_uint16, thd_teorico
34
35 def golden_model(input_signal, taps_fir1, taps_fir2):
36     """Implementa la logica del codigo C en Python"""
37     x = input_signal.astype(float) - 2048.0
38
39     # 1. Filtro Comb
40     y1 = np.zeros_like(x)
41     for n in range(len(x)):
42         delayed_val = y1[n - COMB_DELAY] if n >= COMB_DELAY else 0
43         y1[n] = ALPHA * x[n] + BETA * delayed_val
44
45     # 2. Filtros FIR
46     y2 = np.convolve(y1, taps_fir1, mode='same')
47     y3 = np.convolve(y1, taps_fir2, mode='same')
48
49     # 3. Calculo de THD por bloques
50     thd_output = []
51     for i in range(0, len(y1), BUFFER_SIZE):
52         block_y2 = y2[i:i+BUFFER_SIZE]
53         block_y3 = y3[i:i+BUFFER_SIZE]
54
55         if len(block_y2) < BUFFER_SIZE: break
56
57         v0 = np.sqrt(np.mean(block_y3**2))
58         vt = np.sqrt(np.mean(block_y2**2))
59
60         # THD = sqrt(Vt^2 - V0^2) / V0
61         thd = np.sqrt(np.abs(vt**2 - v0**2)) / v0 if v0 > 1e-6 else 0
62         thd_output.extend([int(thd * 4095)] * len(block_y2))
63
64     return thd_output
65
66 if __name__ == "__main__":
67     # 1. Generar entrada y obtener THD teorico
68     input_data, thd_esperado = generate_inputs("input_samples.txt")
69
70     # 2. Diseno de filtros (usando SAMPLE_RATE definido arriba)
71     fs = SAMPLE_RATE
72
73     # FIR 1: Pasa-todo o banda ancha para Vt
74     n2, beta2, fc2 = 281, 1.8, 530
75     taps_f1 = signal.firwin(n2, fc2, fs=fs, window=('kaiser', beta2))
76
77     # FIR 2: Pasa-bajos para V0 (Fundamental)
78     n3, beta3, fc3 = 281, 2.0, 75
79     taps_f2 = signal.firwin(n3, fc3, fs=fs, window=('kaiser', beta3))
80
81     # 3. Ejecutar modelo
82     golden_result = golden_model(input_data, taps_f1, taps_f2)
83
84     # 4. Calcular THD final obtenido (promedio de la salida del DAC)
85     # Convertimos de escala 0-4095 de vuelta a flotante 0.0-1.0
86     thd_final = np.mean(golden_result) / 4095.0
87
88     # 5. Impresion de resultados
89     print("\n" + "="*45)
90     print(f"{'ANALISIS DE RESULTADOS (GOLDEN MODEL)':^45}")
91     print("="*45)
92     print(f"Frecuencia de Muestreo: {SAMPLE_RATE} Hz")
93     print(f"THD Esperado (Teorico): {thd_esperado:.4f} ({thd_esperado*100:.2f}%)")
94     print(f"THD Final (Procesado): {thd_final:.4f} ({thd_final*100:.2f}%)")
95     print(f>Error absoluto: {abs(thd_esperado - thd_final):.4f}")
96     print("="*45)
97
98     # 6. Guardar referencia
99     np.savetxt("golden_reference.txt", golden_result, fmt='%d')
100    print("Golden Reference generado en 'golden_reference.txt'.")

```

Listing 6: Código empleado para generar input & golden.

donde la verificación allí indicada arroja:

Tabla 4.5: Resultados del Análisis de Distorsión Armónica (Golden Model en Python)

Parámetro	Valor
Frecuencia de Muestreo (f_s)	10000 Hz
THD Esperado (Teórico)	0.3000 (30.00 %)
THD Final (Procesado)	0.2940 (29.40 %)
Error Absoluto	0.0060

Para generar la salida en C se empleo el siguiente código:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5  #include <math.h>
6
7  #define BUFFER_SIZE 1000
8  #define FIR_TAP1 281
9  #define FIR_TAP2 281
10 #define COMB_DELAY 200
11
12 // --- Coeficientes ---
13 const float taps_FIR1[281] = {...};
14 const float taps_FIR2[281] = {...};
15
16 // --- Variables de Estado Globales ---
17 static float state_comb[COMB_DELAY] = {0};
18 static int idx_comb = 0;
19 static float state_y2[FIR_TAP1] = {0};
20 static int idx_y2 = 0;
21 static float state_y3[FIR_TAP2] = {0};
22 static int idx_y3 = 0;
23
24 const float alpha = 0.18181f;
25 const float beta = 0.81818f;
26
27 // --- Prototipos ---
28 void FIR_Process_Buffer(float *in, float *out, int numElem, float *state, int
    *pIdx, const float *taps, int numTaps);
29 float RMS_calculator(float *s, int numElem);
30
31 int main(int argc, char *argv[]) {
32     if (argc < 3) {
33         printf("Uso: %s <archivo_entrada.txt> <archivo_salida.txt>\n", argv
            [0]);
34         return 1;
35     }
36
37     FILE *fin = fopen(argv[1], "r");
38     FILE *fout = fopen(argv[2], "w");
39
40     if (!fin || !fout) {
41         perror("Error al abrir los archivos");
42         return 1;
43     }
44
45     uint16_t adc_buffer[BUFFER_SIZE];
46     float y1[BUFFER_SIZE], y2[BUFFER_SIZE], y3[BUFFER_SIZE];
47
48     printf("Procesando...\n");
49
50     // Bucle principal: lee bloques de BUFFER_SIZE hasta el fin del archivo
51     while (!feof(fin)) {
52         int samples_read = 0;

```

```

53
54 // 1. Leer datos del archivo (asumiendo valores enteros 0-4095)
55 for (int i = 0; i < BUFFER_SIZE && !feof(fin); i++) {
56     if (fscanf(fin, "%hu", &adc_buffer[i]) == 1) {
57         samples_read++;
58     }
59 }
60
61 if (samples_read == 0) break;
62
63 // 2. Procesamiento Comb Filter
64 for (int i = 0; i < samples_read; i++) {
65     float x_n = (float)adc_buffer[i] - 2048.0f;
66     float delayed_y = state_comb[idx_comb];
67     y1[i] = (alpha * x_n) + (beta * delayed_y);
68     state_comb[idx_comb] = y1[i];
69     idx_comb = (idx_comb + 1) % COMB_DELAY;
70 }
71
72 // Procesamiento FIR
73 FIR_Process_Buffer(y1, y2, samples_read, state_y2, &idx_y2, taps_FIR1
74 , FIR_TAP1);
75 FIR_Process_Buffer(y1, y3, samples_read, state_y3, &idx_y3, taps_FIR2
76 , FIR_TAP2);
77
78 // Calculo de THD
79 float V0 = RMS_calculator(y3, samples_read);
80 float Vt = RMS_calculator(y2, samples_read);
81
82 // Evitar division por cero
83 float THD = (V0 > 0.0001f) ? sqrtf(fabsf(Vt * Vt - V0 * V0)) / V0 :
84 0.0f;
85
86 // 5. Generar salida (Escribir el valor de THD mapeado a 12 bits)
87 for (int i = 0; i < samples_read; i++) {
88     uint16_t out_val = (uint16_t)(THD * 4095.0f);
89     fprintf(fout, "%u\n", out_val);
90 }
91
92 fclose(fin);
93 fclose(fout);
94 printf("Proceso completado con exito.\n");
95 return 0;
96 }
97
98 // --- Implementaciones de funciones ---
99
100 void FIR_Process_Buffer(float *in, float *out, int numElem, float *state, int
101 *pIdx, const float *taps, int numTaps) {
102     for (int i = 0; i < numElem; i++) {
103         state[*pIdx] = in[i];
104         float acc = 0.0f;
105         int currIdx = *pIdx;
106         for (int j = 0; j < numTaps; j++) {
107             acc += taps[j] * state[currIdx];
108             if (--currIdx < 0) currIdx = numTaps - 1;
109         }
110         out[i] = acc;
111         *pIdx = (*pIdx + 1) % numTaps;
112     }
113 }
114
115 float RMS_calculator(float *s, int numElem) {
116     float acc = 0.0f;
117     for (int i = 0; i < numElem; i++) {
118         acc += s[i] * s[i];
119     }
120     return sqrtf(acc / (float)numElem);
121 }

```

Listing 7: Código empleado para generar salida de C.

A partir de los archivos .txt generados se pueden comparar los resultados a partir del siguiente código:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def verificar_estable(file_c, file_golden, samples_to_skip=3000):
5     try:
6         # 1. Cargar datos
7         data_c = np.loadtxt(file_c)
8         data_golden = np.loadtxt(file_golden)
9
10        # Sincronizar tamanos
11        min_len = min(len(data_c), len(data_golden))
12
13        # 2. Recortar la parte del transitorio inicial
14        # Saltamos las primeras N muestras (ej: 3000 muestras = 3 bloques)
15        if min_len <= samples_to_skip:
16            print("Error: El archivo es demasiado corto para saltar tantas
17                  muestras.")
18            return
19
20        c_estable = data_c[samples_to_skip:min_len]
21        g_estable = data_golden[samples_to_skip:min_len]
22        muestras = np.arange(samples_to_skip, min_len)
23
24        # 3. Calculo de error en zona estable
25        error = np.abs(c_estable - g_estable)
26        max_err = np.max(error)
27        mean_err = np.mean(error)
28
29        print(f"--- Verificacion (Zona Estable: muestras {samples_to_skip} a
30              {min_len}) ---")
31        print(f"Error maximo: {max_err} LSB")
32        print(f"Error medio: {mean_err:.4f} LSB")
33
34        # 4. Grafica limpia
35        plt.figure(figsize=(12, 7))
36
37        plt.subplot(2, 1, 1)
38        plt.plot(muestras, g_estable, label='Golden Model', color='
39              forestgreen', linewidth=1.5)
40        plt.plot(muestras, c_estable, label='Output C (Estable)', color='
41              crimson', linestyle='--', alpha=0.8)
42        plt.title(f'Comparacion en Regimen Permanente (Saltando {
43              samples_to_skip} muestras)')
44        plt.ylabel('Valor DAC (0-4095)')
45        plt.legend()
46        plt.grid(True, which='both', linestyle='--', alpha=0.5)
47
48        plt.subplot(2, 1, 2)
49        plt.fill_between(muestras, error, color='orange', alpha=0.3, label='
50              Area de Error')
51        plt.plot(muestras, error, color='darkorange', linewidth=1)
52        plt.title('Diferencia Residual (Precision Numerica)')
53        plt.xlabel('Numero de Muestra')
54        plt.ylabel('Error (LSB)')
55        plt.ylim(0, max(50, max_err + 5)) # Zoom en el error
56        plt.legend()
57        plt.grid(True, which='both', linestyle='--', alpha=0.5)
58
59        plt.tight_layout()
60        plt.show()
61
62    except Exception as e:
63        print(f"Error: {e}")
64
65 if __name__ == "__main__":
66     verificar_estable("salida_c.txt", "golden_reference.txt", samples_to_skip
67                      =3000)

```

Listing 8: Código empleado para comparar golden y salida de código en C.

obteniendo así:

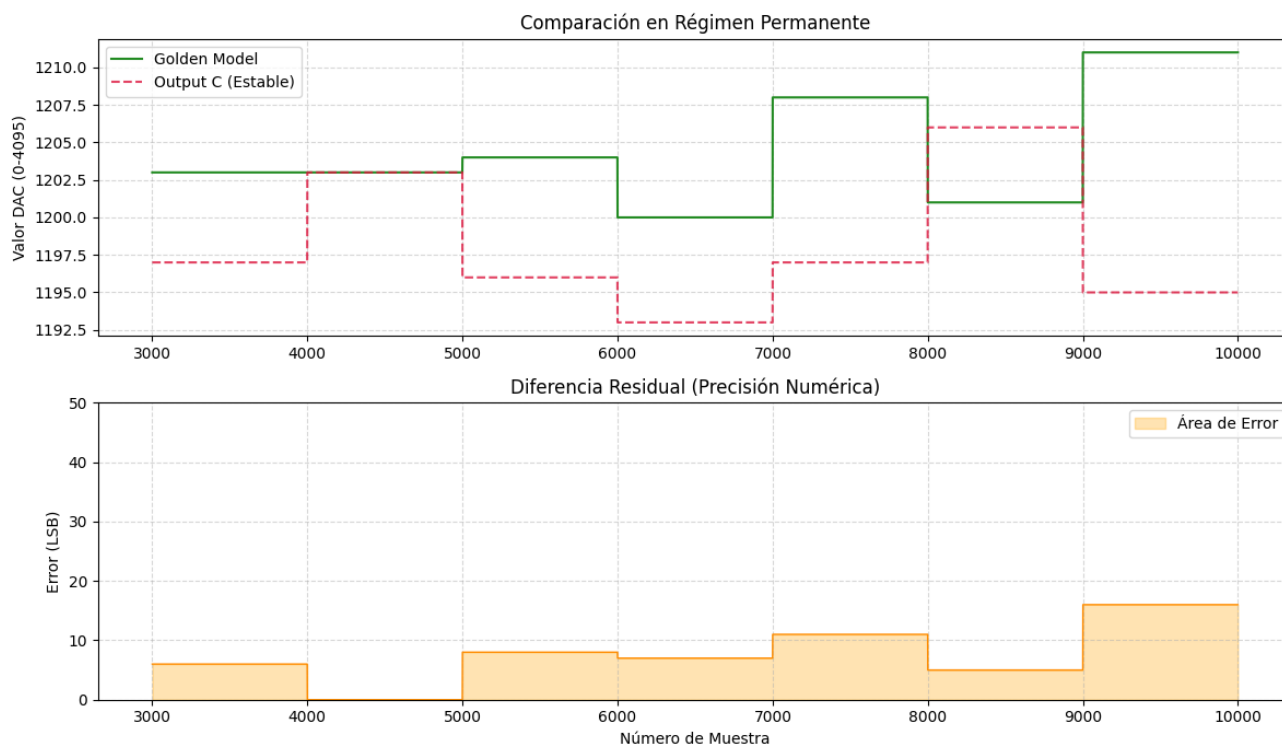


Figura 4.5: Comparativa entre salida en código C y golden model de Python.

donde se observa un error medio de 7.57LSB. De este modo se demuestra que el tratamiento de datos es el adecuado. Sin embargo ha de existir una falencia en la lectura y/o escritura del ADC/DAC respectivamente.